

Guide de Tests SMSI Manager

Stratégie · Outils · Exemples de code · Commandes

Généré le 18/05/2026

Projet	SMSI Manager – Application web de gestion SMSI (ISO 27001)
Stack	.NET 8 (backend) · React (frontend) · SQL Server
Scope	14 modules : Auth, Actifs, Incidents, Risques, PDCA, Cartographie, Clauses, Audits...
Objectif	Tests unitaires, intégration, API REST et End-to-End (E2E)

1 Vue d'ensemble des modules à tester

Le projet SMSI Manager couvre 14 modules fonctionnels. Chaque module doit faire l'objet de tests à plusieurs niveaux (unitaire, intégration, E2E).

Module	Tests prioritaires	Priorité
Auth	Login OK/KO, claims JWT, endpoints protégés	Critique
Users/Roles/Permissions	RBAC lecture/écriture/suppression, périmètre société	Critique
Actifs	CRUD complet, filtres, export CSV, refus sans permission	Haute
Incidents	CRUD, email-import, notifications SignalR/email	Haute
Risques	CRUD études, duplication, owners, isolation société	Haute
PDCA	Cycles/sections/items CRUD	Moyenne
Cartographie	Processus CRUD, liaison clauses/contrôles, upload docs	Moyenne
Clauses/ClauseFile	Conformité, plans d'action, upload/download pièces	Moyenne
Documentation	CRUD, approbation, versioning, téléchargements	Moyenne
Audits	Audits + non-conformités + simulations	Haute
Sensibilisation	Formations CRUD, participants, documents, notify/export	Moyenne
Contrôles	Listing, détail, mise à jour, historique	Moyenne
Dashboard/Traçabilité	Snapshots, export traçabilité, logs d'actions	Basse
Front routing guards	PrivateRoute, PrivateAdminRoute, RequireModulePermission	Haute

2 Outils à installer

2.1 Backend .NET

Les bibliothèques suivantes sont nécessaires pour les tests unitaires et d'intégration .NET :

- xUnit
- Moq
- Microsoft.NET.Test.Sdk
- coverlet.collector
- Microsoft.AspNetCore.Mvc.Testing

2.2 API REST

- Postman (interface graphique)
- Newman (exécution CLI + rapports HTML)

2.3 Frontend React

- Jest + React Testing Library (déjà présents dans le projet)
- msw (Mock Service Worker) – optionnel, pour mocker les appels API

2.4 End-to-End

- Playwright – tests navigateur complets

3 Installation pas à pas

3.1 Vérifier les prérequis

```
dotnet --version
node --version
npm --version
```

3.2 Créer les projets de test backend

```
cd C:\Users\vikram\Desktop\smsi-manager-main\backend
dotnet new xunit -n backend.UnitTests -o tests/backend.UnitTests
dotnet new xunit -n backend.IntegrationTests -o tests/backend.IntegrationTests
dotnet sln backend.sln add tests/backend.UnitTests/backend.UnitTests.csproj
dotnet sln backend.sln add tests/backend.IntegrationTests/backend.IntegrationTests.csproj
dotnet add tests/backend.UnitTests/... reference backend/backend.csproj
dotnet add tests/backend.IntegrationTests/... reference backend/backend.csproj
dotnet add tests/backend.UnitTests/... package Moq
dotnet add tests/backend.UnitTests/... package coverlet.collector
dotnet add tests/backend.IntegrationTests/... package Microsoft.AspNetCore.Mvc.Testing
dotnet add tests/backend.IntegrationTests/... package coverlet.collector
```

3.3 Installer Postman & Newman

```
winget install Postman.Postman
npm install -g newman newman-reporter-htmlextra
```

3.4 Installer Playwright

```
cd C:\Users\vikram\Desktop\smsi-manager-main\frontend
npm install -D @playwright/test
npx playwright install
```

3.5 Lancer tous les tests

```
# Backend
cd ../backend
dotnet test backend.sln --collect:"XPlat Code Coverage"

# Frontend unit
cd ../frontend
$env:CI="true"; npm test -- --watchAll=false

# E2E Playwright
npx playwright test
```

4 Exemples de tests

4.1 xUnit + Moq – Test unitaire (Module Actifs)

Exemple de test du handler de création d'un actif, en mockant le repository :

```
[Fact]
public async Task Handle_Should_Create_Actif_And_Return_Dto()
{
    var repo = new Mock<IActifRepository>();
    var createdId = Guid.NewGuid();

    repo.Setup(r => r.CreateAsync(It.IsAny<Actif>()))
        .ReturnsAsync((Actif a) => { a.Id = createdId; return a; });

    var handler = new CreateActifHandler(repo.Object);
    var cmd = new CreateActifCommand(
        "Serveur AD", "Desc", TypeActif.Support,
        CategorieActif.Infrastructure,
        ClassificationActif.Confidentiel, null, 1);

    var result = await handler.Handle(cmd, CancellationToken.None);

    Assert.Equal(createdId, result.Id);
    Assert.Equal("Serveur AD", result.Nom);
}
```

4.2 Jest + React Testing Library – Garde d'accès (Module Actifs)

Vérification que le composant GestionActifs affiche un message d'erreur si l'utilisateur n'a pas les permissions de lecture :

```
jest.mock('../context/AuthContext', () => ({
    useAuth: () => ({
        canRead: () => false,
        canWrite: () => false,
        canEdit: () => false,
        canDelete: () => false,
        canExport: () => false
    })
}));

test('affiche acces non autorise si pas de permission lecture', () => {
    render(<GestionActifs />);
    expect(screen.getByText(/acces non autorise/i)).toBeInTheDocument();
});
```

4.3 Postman – Tests API (Module Incidents)

Script de test à ajouter dans l'onglet Tests d'une requête Postman :

```
pm.test("Status 200", function () {
    pm.response.to.have.status(200);
});

pm.test("incidentId present", function () {
    const json = pm.response.json();
```

```

    pm.expect(json.incidentId).to.exist;
  });

  pm.test("Champs obligatoires presents", function () {
    const json = pm.response.json();
    pm.expect(json.titre).to.be.a('string');
    pm.expect(json.statut).to.be.a('string');
  });

```

4.4 Playwright – Test E2E (Login + Création actif)

Test complet du flux utilisateur : connexion, navigation vers le module Actifs, création d'un actif :

```

test('login puis creation actif', async ({ page }) => {
  await page.goto('http://localhost:3000/login');
  await page.locator('input[type="email"]')
    .fill(process.env.E2E_EMAIL!);
  await page.locator('input[type="password"]')
    .fill(process.env.E2E_PASSWORD!);
  await page.getByRole('button', { name: /se connecter/i }).click();

  await page.goto('http://localhost:3000/actifs');
  await page.getByRole('button', { name: /nouveau actif/i }).click();
  await page.getByPlaceholder("Nom de l'actif").fill('Actif E2E');
  await page.getByRole('button', { name: /enregistrer/i }).click();

  await expect(page.getByText('Actif E2E')).toBeVisible();
});

```

5. Commandes de lancement (résumé)

Type	Commande
Tests backend .NET	<code>dotnet test backend.sln --collect:"XPlat Code Coverage"</code>
Tests frontend unit	<code>\$env:CI="true"; npm test -- --watchAll=false</code>
Tests E2E	<code>npx playwright test</code>
Rapport Playwright	<code>npx playwright show-report</code>
Newman (Postman CLI)	<code>newman run SMSI_API.postman_collection.json -e SMSI_Local.postman_environment.json -r cli,htmlextra --reporter-htmlextra-export ./tests/reports/newman/report.html</code>

6. Fichiers à créer et modifier

Fichiers à créer

- `backend/tests/backend.UnitTests/` (projet xUnit unitaire)
- `backend/tests/backend.IntegrationTests/` (projet xUnit intégration)
- `frontend/playwright.config.ts` (configuration Playwright)
- `frontend/tests/e2e/*.spec.ts` (scénarios E2E)
- `tests/postman/SMSI_API.postman_collection.json`
- `tests/postman/SMSI_Local.postman_environment.json`

Fichiers à modifier

Fichier	Modification requise
<code>backend/backend.sln</code>	Ajouter les deux nouveaux projets de tests
<code>backend/Program.cs</code>	Ajouter <code>public partial class Program { }</code> pour les tests d'intégration
<code>frontend/package.json</code>	Ajouter les scripts <code>test:unit</code> et <code>test:e2e</code>
<code>frontend/src/App.test.js</code>	Mettre à jour le test obsolète (erreur Router context)

7. Captures et résultats attendus pour le rapport

Élément	Détail attendu
État initial frontend	PASS url.test.js — FAIL App.test.js (erreur Router context manquant)
État initial backend	Échec dotnet test : NU1301 (problème accès NuGet/SSL)
Postman Collection Runner	Capture du nombre de tests pass/fail par endpoint
Newman HTML report	Fichier généré : tests/reports/newman/report.html
Playwright HTML report	Généré avec npx playwright show-report
Couverture backend	Rapport XPlat Code Coverage converti en HTML avec reportgenerator

8. Point critique détecté : AuthContext / refresh token

■ *AuthContext.jsx appelle /api/auth/refresh, mais aucun endpoint refresh n'a été trouvé côté API backend. C'est un cas de test important à documenter.*

Ce bug potentiel doit être couvert par les tests suivants :

- Test unitaire frontend : mocker l'appel /api/auth/refresh et vérifier le comportement en cas d'erreur 404
- Test d'intégration backend : vérifier l'absence ou la présence intentionnelle de l'endpoint refresh
- Test E2E : vérifier que la session expire correctement et redirige vers /login

Scénarios de test suggérés

```
// Jest - AuthContext refresh manquant
test('redirige vers login si refresh echoue', async () => {
  server.use(
    rest.post('/api/auth/refresh', (req, res, ctx) =>
      res(ctx.status(404))
    )
  );
  render(<App />);
  await waitFor(() =>
    expect(window.location.pathname).toBe('/login')
  );
});
```

9. Plan de mise en place sur 3 jours

Jour	Phase	Tâches
Jour 1	Backend	Créer projets xUnit · Écrire tests unitaires Auth + Actifs + Risques · Lancer dotnet test · Capturer couverture
Jour 2	API & Frontend	Créer collection Postman (Auth, Actifs, Incidents) · Lancer Newman · Mettre à jour App.test.js · Écrire tests RTL (gardes d'accès) · Capturer résultats

Jour	Phase	Tâches
Jour 3	E2E & Rapport	Configurer Playwright · Écrire 3-5 scénarios E2E · Lancer npx playwright test · Compiler toutes les captures dans le rapport final

Guide généré automatiquement à partir de l'analyse de la structure réelle du projet SMSI Manager.